

Lecture 9

Transaction Management

Dr. Vandana Kushwaha

Department of Computer Science
Institute of Science, BHU, Varanasi

Introduction

- **Collections of operations** that form a **single logical unit of database processing** are called **Transactions**.
- A **transaction** includes one or more **database access operations**—these can include **insertion, deletion, modification**, or **retrieval** operations.
- **Read-only transaction**: a transaction **do not update** the **database** but **only retrieve** data.
- **Read-write transaction**: a transaction **update** the **database**.
- To **ensure integrity** of the data, we require that the **database system** maintain the following **properties of the transactions**:
 - **Atomicity**
 - **Consistency**
 - **Isolation**
 - **Durability**

Properties of Transaction

- **ATOMICITY**
- **Either** all operations of the **transaction** are **reflected properly** in the **database**, or **none** are.
- **CONSISTENCY**
- Execution of a **transaction** in **isolation** (that is, with no other transaction executing concurrently) **preserves the consistency** of the **database**.

Properties of Transaction

- **ISOLATION**

- Even though multiple transactions may execute **concurrently**, the system guarantees that, for every pair of transactions T_i and T_j ,
 - It appears to T_i that either T_j finished execution before T_i started, or
 - T_j started execution after T_i finished.
 - Thus, each transaction is **unaware** of other transactions executing **concurrently** in the system.

- **DURABILITY**

- After a transaction **completes successfully**, the **changes** it has made to the database **persist**, even if there are **system failures**.
- **Note:** These **properties** of transactions are also known as **ACID properties** of transaction.

Properties of Transaction: Example

- Consider a simplified **banking system** consisting of **several accounts** and a set of **transactions** that **access** and **update** those **accounts**.
- We assume that the **database** permanently resides on **disk**, but that **some portion** of it is **temporarily residing** in **main memory buffer**.
- **A Transactions** can access **data** using **two operations**:
 - **read(X)**, which **transfers the data item X** from the **database**(in hard disk) to a **local buffer**(in main memory) belonging to the **transaction** that executed the read operation.
 - **write(X)**, which **transfers the data item X** from the **local buffer**(in main memory) of the **transaction** that executed the **write back** to the **database** (in hard disk) .

Properties of Transaction: Example

- Let ***T_i*** be a **Transaction** that transfers \$50 from **account *A*** to **account *B***.
- This **transaction *T_i*** can be defined as:
 - ***T_i*: read(*A*);**
 - ***A* := *A* - 50;**
 - **write(*A*);**
 - **read(*B*);**
 - ***B* := *B* + 50;**
 - **write(*B*)**

ATOMICITY : Example

- Suppose that, just **before the execution** of transaction *T_i* the values of **accounts A** and **B** are **\$1000** and **\$2000**, respectively.
- Now suppose that, during the **execution** of transaction *T_i*, a **failure occurs** that prevents *T_i* from completing its execution successfully.
- **Examples** of such failures include **power failures, hardware failures, and software errors.**
- Further, suppose that the **failure happened after the write(A)** operation but **before the write(B)** operation.
- In this case, the values of **accounts A** and **B** reflected in the **database** are **\$950** and **\$2000.**
- The system **destroyed \$50** as a result of this **failure.**

ATOMICITY : Example

- In particular, we note that the sum **$A + B$** is no longer **preserved**.
- The **basic idea** behind **ensuring atomicity** is this:
 - The **database system** keeps track (on disk) of the **old values** of any **data** on which a **transaction** performs a **write**.
- And, if the **transaction** does not complete its **execution**, the **database system** **restores the old values** to make it **appear** as though the **transaction** never **executed**.
- Ensuring **Atomicity** is handled by a component called the **Transaction-Management Component** of **DBMS** software .

Properties of Transaction: CONSISTENCY

- The **Consistency** requirement here is that the **sum of A and B** be **unchanged** by the **execution** of the **transaction**.
- **Without the consistency** requirement, **money** could be **created or destroyed** by the **transaction**!
- Ensuring **consistency** for an **individual transaction** is the **responsibility** of the **Application programmer** who codes the **transaction**.

CONSISTENCY : Example 1

- Let the initial values of **A** and **B** are **1000** and **2000** respectively.

T1	Database state
read(A)	A=1000
A=A-50	A=1000-50
write(A)	A=950
read(B)	B=2000
B=B+50	B=2000+50
write(B)	B=2050

- Value of **A+B** before executing the transaction $T1=1000+2000=$ **3000**
- Value of **A+B** after executing the transaction $T1=950+2050=$ **3000**
- Thus transaction **T1** **satisfy** the **Consistency** property.

CONSISTENCY : Example 2

- Let the initial values of **A** and **B** are **1000** and **2000** respectively.

T1	T2	Database state
read(A) A=A-50 write(A) Read(B) B=B+50 Write(B)		A=1000 A=1000-50 A=950 B=2000 B=2000+50 B=2050
	read(A) temp= A*0.1 A= A – temp write(A) Read(B) B=B + temp write(B)	A=950 temp=950*0.1 A=950 – 95 A= 855 B=2050 B=2050 + 95 B=2145

- Value of **A+B** before executing the transaction T1 and T2 = $1000+2000=$ **3000**
- Value of **A+B** after executing the transaction T1 and T2 = $855+2145=$ **3000**
- Thus transaction **T1** and **T2** **satisfy** the **Consistency** property.

CONSISTENCY : Example 3

T1	T2	Database state
read(A) A=A-50		A=1000 A=1000-50
	read(A) temp= A*0.1 A= A – temp write(A) Read(B)	A=1000 temp=1000*0.1 A=1000-100 A=900 B=2000
write(A) Read(B) B=B+50 write(B)		A=950 B=2000 B=2000+50 B=2050
	B=B+temp write(B)	B=2000+100 B=2100

- Let the initial values of **A** and **B** are **1000** and **2000** respectively.
- Value of **A+B** before executing the transaction T1 and T2 = $1000+2000=3000$
- Value of **A+B** after executing the transaction T1 and T2 = $950+2100=3050$
- Thus transaction **T1** and **T2** **does not** satisfy the **Consistency** property.

CONSISTENCY : Example 4

T1	T2	Database state
read(A) A=A-50		A=1000 A=1000-50
	read(A) temp= A*0.1 A= A – temp write(A) Read(B) B=B+temp write(B)	A=1000 temp=1000*0.1 A=1000-100 A=900 B=2000 B=2000+100 B=2100
write(A) Read(B) B=B+50 write(B)		A=950 B=2100 B=2100+50 B=2150

- Let the initial values of **A** and **B** are **1000** and **2000** respectively.
- Value of **A+B** before executing the transaction T1 and T2 = $1000+2000=$ **3000**
- Value of **A+B** after executing the transaction T1 and T2 = $950+2150=$ **3100**
- Thus transaction **T1** and **T2** **does not** satisfy the **Consistency** property.

Properties of Transaction: ISOLATION

- If **several transactions** are **executed concurrently**, their **operations** may **interleave** in some **undesirable way**, resulting in an **inconsistent database state**.
- For **example** the **database** is temporarily **inconsistent** while the transaction to transfer funds from **A** to **B** is executing, with the **deducted** total **written** to **A** and the **increased** total **yet to be written** to **B**.
- If a **second** concurrently running transaction reads **A** and **B** at this **intermediate point** and computes **$A+B$** , it will observe an **inconsistent value**.
- Furthermore, if this **second transaction** then performs **updates** on **A** and **B** based on the **inconsistent values** that it read, the **database** may be left in an **inconsistent state** even after both transactions have **completed**.

ISOLATION: Example

T1	T2	Database state
read(A) A=A-50 write(A)		A=1000 A=1000-50 A=950
	read(A) read(B) Display(A+B)	A=950 B=2000 A+B=2950
read(B) B=B+50 write(B)		B=2000 B=2000+50 B=2050

- Let the initial values of **A** and **B** are **1000** and **2000** respectively.
- Transaction **T2** display the **wrong value** of **A+B**. Thus transaction **T2** is in **inconsistent** state.

Properties of Transaction: ISOLATION

- A way to avoid the problem of **concurrently executing transactions** is to execute transactions **serially**—that is, one after the other.
- However, **concurrent execution** of transactions provides **significant performance benefits**.
- The **Isolation property** of a transaction **ensures** that the **concurrent execution** of transactions results in a **system state** that is **equivalent** to a **state** that could have been obtained had these transactions executed one at a time in some order i.e. **serially**.
- Ensuring the **isolation property** is the **responsibility** of a **component** of the **database system** called the **Concurrency-control component**.

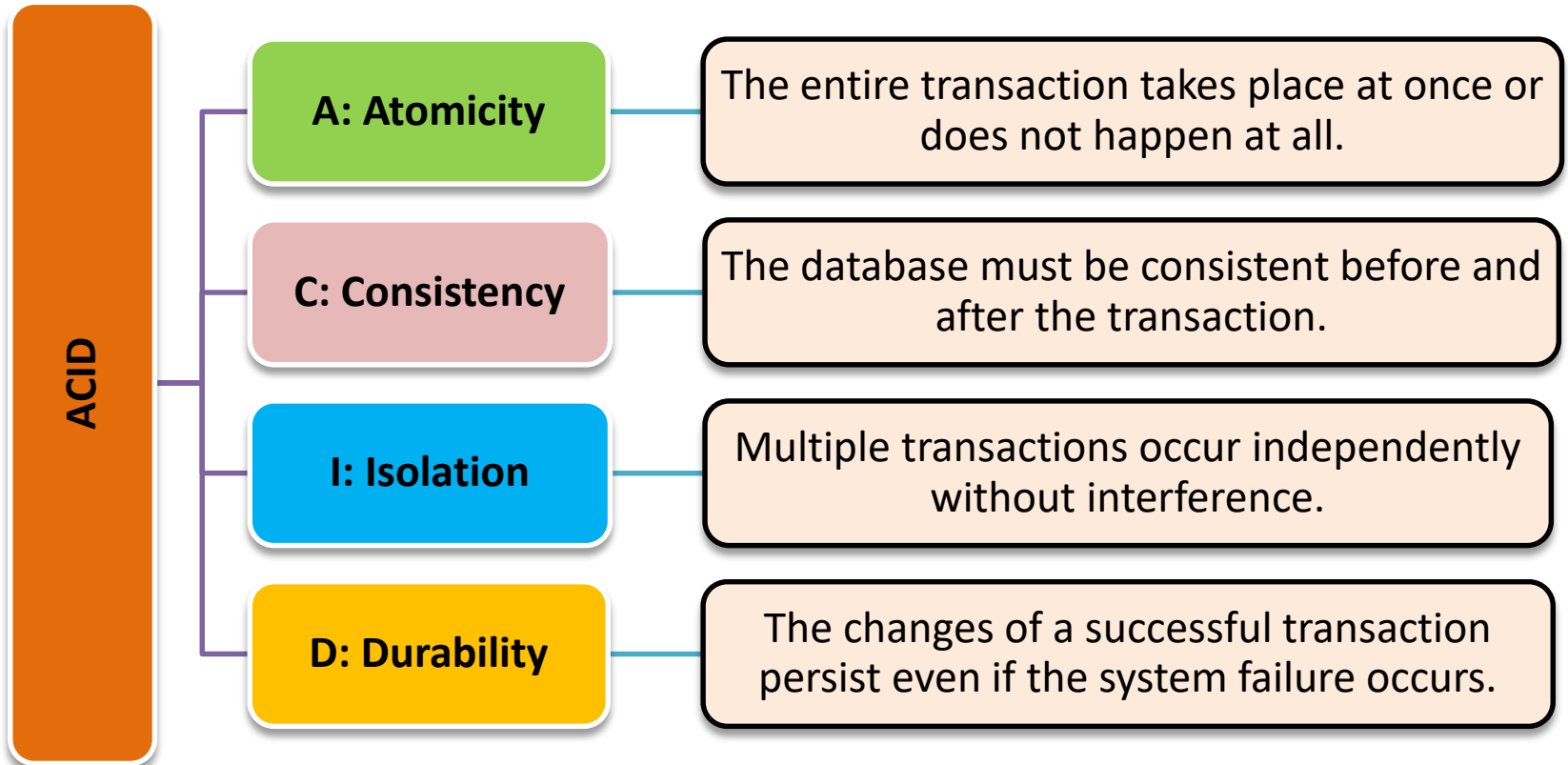
Properties of Transaction: DURABILITY

- Once the **execution** of the **transaction** **completes successfully**, and the user who initiated the **transaction** has been **notified** that the **transfer of funds** has taken place.
- It **must** be the case that **no system failure** will **result in a loss of data** corresponding to this **transfer of funds**.
- The **Durability property** guarantees that, once a **transaction completes successfully**, all the **updates** that it carried out on the **database** **persist**, even if there is a **system failure** after the **transaction completes** execution.

Properties of Transaction: DURABILITY

- We assume that a **failure** of the computer system may result in **loss of data** in **main memory**, but **data** written to **disk** are **never lost**.
- We can guarantee **Durability** by ensuring that either:
 - The **updates** carried out by the **transaction** have been **written to disk** before the **transaction completes**.
 - **Information** about the **updates** carried out by the **transaction** written to **disk** is **sufficient** to enable the **database** to **reconstruct the updates** when the **database system** is **restarted** after the **failure**.
- Ensuring **Durability** is the **responsibility** of a **component** of the database system called the **Recovery-management component**.

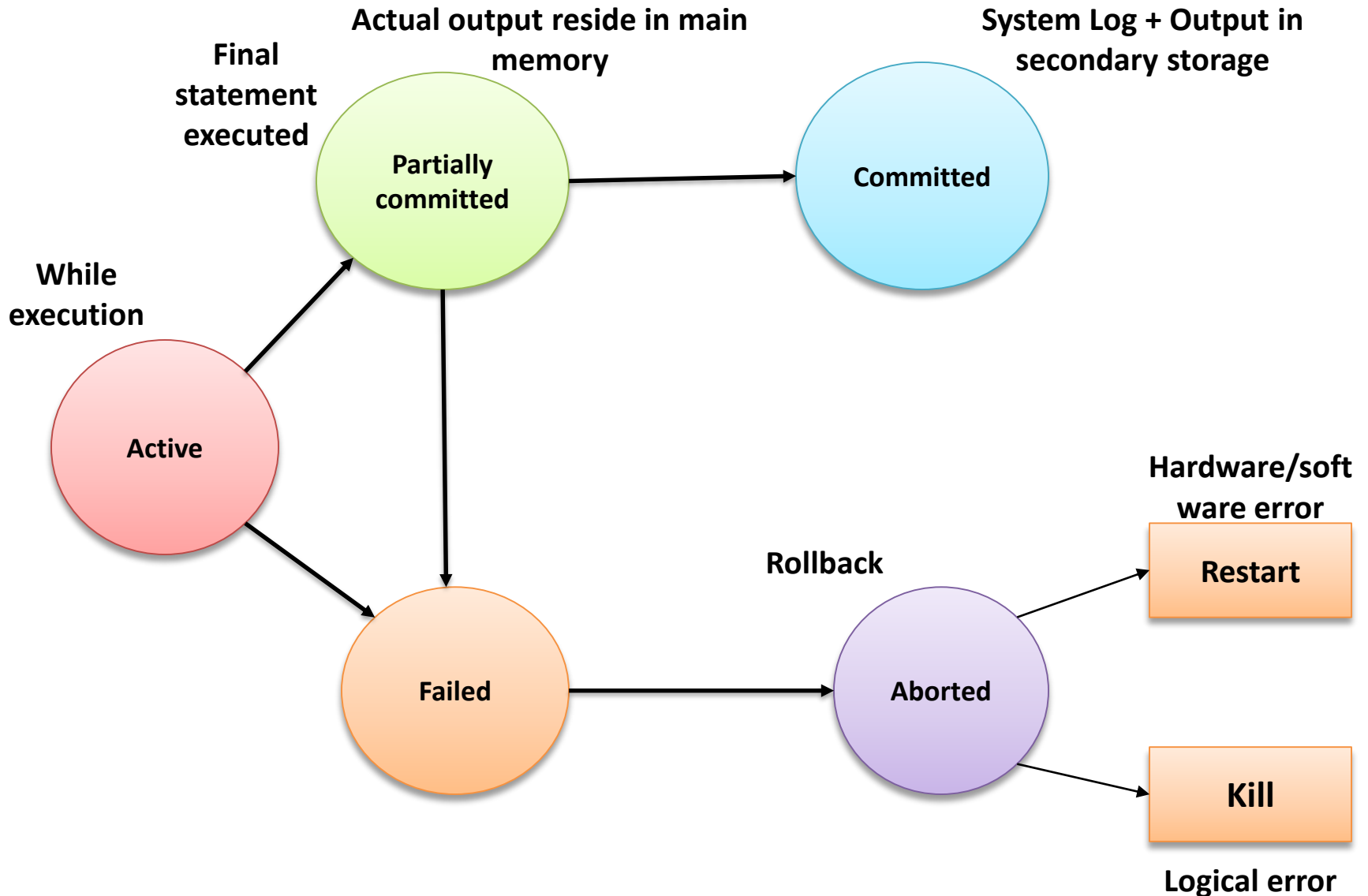
Properties of Transaction: Summary



Transaction States

- A **Transaction** must be in **one** of the following **states**:
- **Active**
 - The **initial state**; the **transaction** stays in this **state** while it is **executing**.
- **Partially committed**
 - After the **final statement** has been **executed**.
- **Failed**
 - After the discovery that normal execution can no longer proceed.
- **Aborted**
 - After the transaction has been **rolled back** and the **database** has been **restored** to its **state** prior to the **start** of the **transaction**.
- **Committed**
 - After **successful completion**.

Transaction States



Transaction States

- A **Transaction** starts in the **Active state**.
- When it **finishes** its **final statement**, it enters the **Partially committed state**.
 - At this point, the **transaction** has **completed** its **execution**,
 - But it is **still possible** that it may have to be **aborted**, since the **actual output** may still be **temporarily residing** in **main memory**, and
 - Thus a **hardware failure** may **preclude** its **successful completion**.
- The **database system** then **writes out** enough information to **disk(log file)** that, even in the event of a **failure**, the **updates** performed by the **transaction** can be **re-created** when the **system restarts** after the **failure**.
- When **the last** of this information is **written** out on the **disk(log file)** , the **transaction** enters the **Committed state**.

Transaction States

- A **Transaction** enters the **Failed state** after the system determines that the **transaction** can **no longer proceed** with its **normal execution** (for example, because of **hardware** or **logical errors**).
- Such a **transaction** must be **Rolled back**, then, it enters the **Aborted state**. At this point, the system has **two options**:
 - **Restart the transaction**
 - If the **transaction** was **aborted** as a result of **some hardware** or **software error**.
 - A **restarted transaction** is considered to be a **new transaction**.
 - **Kill the transaction**
 - It usually does so because of some **internal logical error** that can be corrected only by **rewriting** the **application program**, or because the **input was bad**, or because the **desired data** were **not found** in the **database**.

The System Log

- To be able to **recover from failures** that affect **transactions**, the system maintains a **log** to keep track of all transaction operations that affect the **values** of database items.
- The **log** is a **sequential, append-only file** that is **kept on disk**, so it is **not affected** by any type of **failure** except for **disk** or **catastrophic failure**.
- Typically, one (or more) **main memory buffers** hold the **last part of the log file**, so that **log entries** are **first added** to the **main memory buffer**.
- When the **log buffer** is **filled**, the **log buffer** is *appended to the end of the log file on disk*.
- In addition, the **log file** from **disk** is **periodically backed up** to **archival storage (tape)** to **guard** against **catastrophic failures**.
- The following are the **types of entries**—called **log records**—that are **written** to the **log file** and the corresponding action for each **log record**:

The System Log

- **[start_transaction, T].**

Indicates that transaction T has started execution.

- **[write_item, T, X, old_value, new_value].**

Indicates that transaction T has **changed** the value of database item X from *old_value* to *new_value*.

- **[read_item, T, X].**

Indicates that transaction T has **read** the value of database item X .

- **[commit, T].**

Indicates that transaction T has **completed successfully**, and affirms that its effect can be committed (recorded permanently) to the database.

- **[abort, T].**

Indicates that transaction T has been **aborted**.

Commit Point of a Transaction

- A transaction T reaches its **Commit point** when:
 - All its operations that access the database have been executed successfully *and*
 - the effect of all the transaction operations on the database have been recorded in the **log file**.
- Beyond the **Commit point**, the Transaction is said to be **Committed**, and its effect must be *permanently recorded* in the database.
- The transaction then writes a commit record **[commit, T]** into the **log**.

Commit Point of a Transaction

- If a **system failure** occurs:
 - We can **search back in the log** for all transactions T that have **written** a **[start_transaction, T]** record into the **log** but have **not written** their **[commit, T]** record yet;
 - These **transactions** may have to be **rolled back** to **undo their effect** on the **database** during the **recovery process**.
 - **The Transactions** that have **written** their **Commit record [commit, T]** in the **log** must also have **recorded** all their **WRITE operations in the log**, so their **effect** on the **database** can be **redone** from the **log records**.

Serial Execution

- The **execution sequences** of transactions are called **schedules**, shown below:

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

Schedule 1

A serial schedule in which T_2 is followed by T_1 .

- The final values of accounts **A** and **B**, after the execution takes place, are **\$855** and **\$2145**, respectively.
- Thus, the **total amount of money** in accounts **A** and **B**—that is, the **sum $A + B$** is **preserved** after the execution of both **transactions**.

Serial Execution

- A **serial schedule** in which **T1** is followed by **T2**.

T_1	T_2
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

Schedule 2

A serial schedule in which **T1** is followed by **T2**.

- Check the **sum $A + B$** is **preserved** or not?
- These schedules(Schedule 1 and Schedule 2) are **serial**.
- Each **serial schedule** consists of a **sequence of instructions** from various transactions, where the instructions belonging to one single transaction appear together in that schedule.
- Thus, for a **set of n transactions**, there exist **$n!$ different valid serial schedules**.

Concurrent Executions

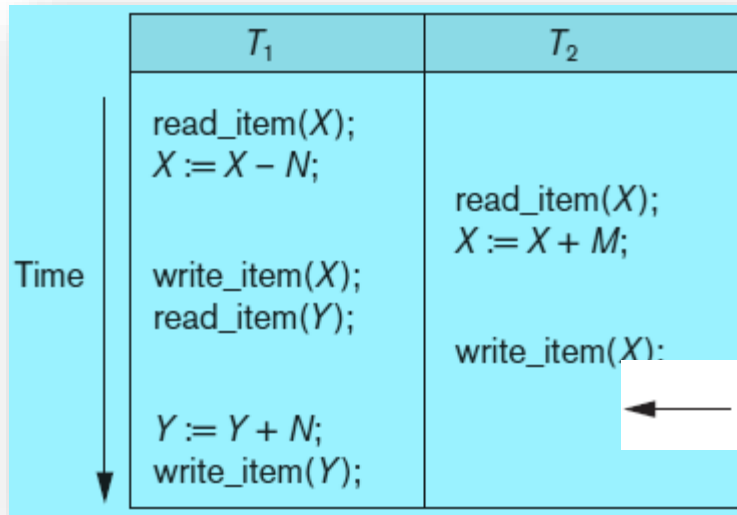
- **Transactions** submitted by the various users may **execute concurrently** and may **access** and **update** the **same database items**.
- If this **concurrent execution** is **uncontrolled**, it may lead to **problems**, such as an **inconsistent database**.
- **Example:** Consider two **transactions T1** and **T2**:

T_1
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>

T_2
<pre>read_item(X); X := X + M; write_item(X);</pre>

Problems with Concurrent Executions

- **The Lost Update Problem**
- This **problem** occurs when **two transactions** that access the **same database items** have their **operations interleaved** in a way that makes the value of some **database items incorrect**.
- Suppose that **transactions T_1 and T_2** are **submitted** at approximately the **same time**, and suppose that their operations are **interleaved** as shown in Figure:



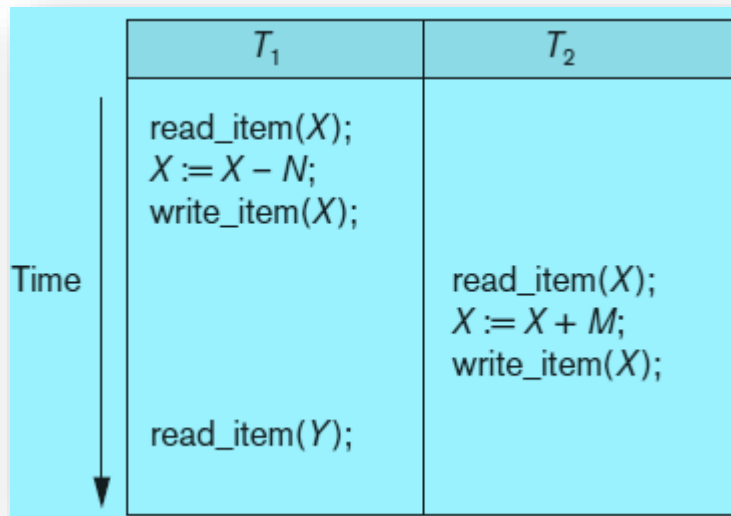
- if $X = 80$ at the start, $N = 5$ and $M = 4$ the final result should be $X = 79$.
- However, in the interleaving of operations it is $X = 84$.
- Because the update in **T_1** that deducted five from **X** was **lost**.

← Item X has an incorrect value because its update by T_1 is *lost* (overwritten).

Temporary Update/Dirty Read Problem

- This **problem** occurs when one **transaction updates** a **database** item and then the **transaction fails** for some reason.
- Meanwhile, the **updated item** is **accessed (read)** by **another transaction** before it is changed back to its **original value**.
- Figure shows an **example** where **T1 updates item X** and then **fails before completion**, so the **system** must change **X** back to its **original value**.
- Before it can do so, however, **transaction T2 reads the temporary value** of **X**, which will not be recorded permanently in the database because of the failure of **T1**.
- The value of **item X** that is read by **T2** is called **dirty data** because it has been created by a transaction that has not completed and **committed** yet; hence, this problem is also known as the **dirty read problem**.

Temporary Update/Dirty Read Problem



Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X .

The Incorrect Summary Problem

- If one **transaction** is calculating an **aggregate summary** function on a number of database items.
- While other **transactions** are **updating** some of these items, the **aggregate function** may calculate some values before they are updated and others after they are updated.

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ ⋮ ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

Concurrent Execution

- If **two transactions** are running **concurrently**, the **operating system** may execute **one transaction** for a **little while**, then perform a **context switch**, execute the **second transaction** for some time, and then **switch back** to the **first transaction** for some time, and so on.
- With **multiple transactions**, the **CPU time** is **shared** among all the **transactions**.
- Several **execution sequences** are **possible**, since the various **instructions** from **both transactions** may now be **interleaved**.
- Thus, the **number of possible schedules** for a set of **n transactions** is **much larger than $n!$** .
- **Example:** Let **two transactions** are executed **concurrently**.
- One possible schedule is **Schedule 3** shown in figure:

Concurrent Execution

T ₁	T ₂
read(<i>A</i>) <i>A</i> := <i>A</i> - 50 write(<i>A</i>)	read(<i>A</i>) <i>temp</i> := <i>A</i> * 0.1 <i>A</i> := <i>A</i> - <i>temp</i> write(<i>A</i>)
read(<i>B</i>) <i>B</i> := <i>B</i> + 50 write(<i>B</i>)	read(<i>B</i>) <i>B</i> := <i>B</i> + <i>temp</i> write(<i>B</i>)

Schedule 3—a concurrent schedule equivalent to Schedule 1.

- After this execution takes place, we arrive at the **same state** as the one in which the **transactions** are **executed serially** in the **order T1 followed by T2**.
- The **sum *A* + *B*** is indeed **preserved**.

Concurrent Execution

- Not all **concurrent executions** result in a **correct state**.
- Consider the schedule of Figure given below:

T_1	T_2
read(A) $A := A - 50$	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B)
write(A) read(B) $B := B + 50$ write(B)	 $B := B + temp$ write(B)

Schedule 4—a concurrent schedule.

- After the execution of this schedule, we arrive at a **state** where the **final values** of accounts A and B are **\$950** and **\$2100**, respectively.
- This **final state** is an **inconsistent state**, since we have **gained \$50** in the process of the **concurrent execution**.
- Indeed, the **sum $A + B$** is **not preserved** by the execution of the two transactions.

Concurrent Execution

- We can ensure **consistency** of the **database** under **concurrent execution** by making sure that **any concurrent schedule** that executed has the **same effect** as a **schedule** that could have **occurred without any concurrent execution**.
- That is, *the **concurrent schedule** should, in some sense, be **equivalent to a serial schedule**.*
- **Serializable schedule**
- A **schedule S** of **n** transactions is **serializable** if it is **equivalent** to some **serial schedule** of the same **n** transactions.
- There are **two types** of **serializability**:
 1. **Conflict Serializability**
 2. **View Serializability**

Conflict Serializability

- Let us consider a **schedule S** in which there are **two consecutive** instructions ***li*** and ***lj***, of transactions ***Ti*** and ***Tj***, respectively.

Schedule S	
Ti	Tj
li	lj

- If ***li*** and ***lj*** refer to **different data items**, then we can **swap *li* and *lj*** without affecting the results of any instruction in the schedule. **Example:**

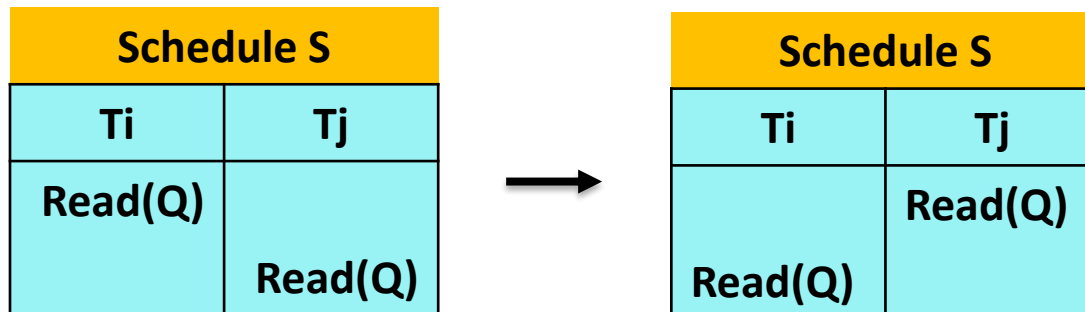
Schedule S	
Ti	Tj
Read(Q)	Write(R)



Schedule S	
Ti	Tj
Read(Q)	Write(R)

Conflict Serializability

- However, if li and lj refer to the **same data item Q** , then the **order** of the two steps may **matter**.
- There are **four cases** that we need to consider:
- **Case 1: $li = \text{read}(Q)$, $lj = \text{read}(Q)$.**
 - The order of li and lj does **not matter**.
 - Since the same value of Q is read by Ti and Tj , regardless of the order.



Conflict Serializability

- **Case 2: $li = \text{read}(Q)$, $lj = \text{write}(Q)$.**

- If **li** comes before **lj** , then **Ti** does not read the value of **Q** that is written by **Tj** in instruction **lj** .

Schedule S	
T_i	T_j
Read(Q)	Write(Q)

Schedule S	
T_i	T_j
Read(Q)	Write(Q)

- If **lj** comes before **li** , then **Ti** reads the value of **Q** that is written by **Tj** .
- Thus, the **order of li and lj matters**.

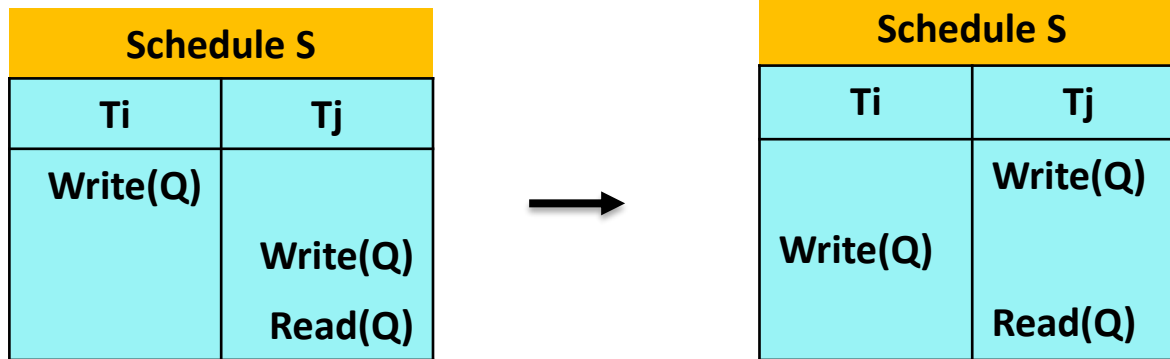
- **Case 3: $li = \text{write}(Q)$, $lj = \text{read}(Q)$.**

- The order of **li** and **lj** matters for reasons similar to those of the **previous case**.

Conflict Serializability

- Case 4: $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$.

- Since both instructions are **write** operations, the **order** of these instructions **does not affect** either T_i or T_j .



- However, the value obtained by the next **read(Q)** instruction of S is **affected**, since the result of only the **latter** of the **two write instructions** is **preserved** in the **database**.
- If there is no other **write(Q)** instruction after I_i and I_j in S , then the order of I_i and I_j directly affects the **final value** of Q in the **database state** that results from **schedule S**.

Conflict Serializability

- Thus, only in the case where **both I_i and I_j are read instructions** does the relative **order of their execution not matter**.
- We say that I_i and I_j **conflict** if they are **operations by different transactions on the same data item**, and **at least one of these instructions is a write operation**.

Conflict Serializability

- Consider **Schedule 3**:
- The **write(A)** instruction of **T1** **conflicts** with the **read(A)** instruction of **T2**.
- However, the **write(A)** instruction of **T2** does **not conflict** with the **read(B)** instruction of **T1**, because the two instructions access different data items.
- Let **li** and **lj** be **consecutive instructions** of a **schedule S**.
- If **li** and **lj** are instructions of **different transactions** and **li** and **lj** **do not conflict**, then **we can swap the order of li and lj** to produce a new **schedule S'**.
- Since the **write(A)** instruction of **T2** in **Schedule 3** does not conflict with the **read(B)** instruction of **T1**, we can **swap** these instructions to generate an equivalent schedule, **schedule 5**.

T ₁	T ₂
read(A) $A := A - 50$ write(A)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A)
read(B) $B := B + 50$ write(B)	
	read(B) $B := B + temp$ write(B)

T ₁	T ₂
read(A) write(A)	read(A)
read(B)	write(A)
write(B)	read(B) write(B)

Conflict Serializability

- We continue to **swap non-conflicting instructions**:
 - Swap the `read(B)` instruction of T_1 with the `read(A)` instruction of T_2 .
 - Swap the `write(B)` instruction of T_1 with the `write(A)` instruction of T_2 .
 - Swap the `write(B)` instruction of T_1 with the `read(A)` instruction of T_2 .
- The final result of these swaps, **schedule 6**.

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

- Thus, we have shown that **schedule 3** is **equivalent** to a **serial schedule**.
- This **equivalence** implies that, regardless of the initial system state, **schedule 3** will **produce the same final state** as will some **serial schedule**.

Conflict Serializability

- If a **schedule S** can be **transformed** into a **schedule S'** by a **series of swaps of non-conflicting instructions**, we say that **S and S'** are **conflict equivalent**.
- In our previous examples, **Schedule 1** is **not conflict equivalent** to **Schedule 2**.
- However, **Schedule 1** is **conflict equivalent** to **Schedule 3**.

T ₁	T ₂
read(A) A := A - 50 write(A) read(B) B := B + 50 write(B)	read(A) temp := A * 0.1 A := A - temp write(A) read(B) B := B + temp write(B)

Schedule 1

T ₁	T ₂
read(A) A := A - 50 write(A) read(B) B := B + 50 write(B)	read(A) temp := A * 0.1 A := A - temp write(A) read(B) B := B + temp write(B)

Schedule 2

T ₁	T ₂
read(A) A := A - 50 write(A) read(B) B := B + 50 write(B)	read(A) temp := A * 0.1 A := A - temp write(A) read(B) B := B + temp write(B)

Schedule 3

Conflict Serializability

- The concept of **conflict equivalence** leads to the concept of **conflict serializability**.
- We say that a concurrent ***schedule S is conflict serializable if it is conflict equivalent to a serial schedule.***
- Thus, **Schedule 3** is **conflict serializable**, since it is **conflict equivalent** to the serial **Schedule 1**.

T ₁	T ₂
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

Schedule 1

T ₁	T ₂
read(A) $A := A - 50$ write(A)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A)
read(B) $B := B + 50$ write(B)	read(B) $B := B + temp$ write(B)

Schedule 3

Conflict Serializability

- Finally, consider **Schedule 7** of Figure:

T_3	T_4
read(Q)	write(Q)
write(Q)	

- It consists of only the significant operations (that is, the read and write) of transactions **T_3** and **T_4** .
- This schedule is **not conflict serializable**, since it is not equivalent to either the serial schedule $\langle T_3, T_4 \rangle$ or the serial schedule $\langle T_4, T_3 \rangle$.

Important Note

- It is possible to have **two schedules** that produce the **same outcome**, but that are **not conflict equivalent**.
- For **example**, consider **transaction T_5** , which transfers **\$10** from account B to account A.

Conflict Serializability

- Let **Schedule 8** be as defined in Figure.

T_1	T_5
read(A)	
$A := A - 50$	
write(A)	
	read(B)
	$B := B - 10$
	write(B)
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$A := A + 10$
	write(A)

Schedule 8

- We claim that **Schedule 8** is **not conflict equivalent** to the **serial schedule**, since, in **Schedule 8**, the **write(B)** instruction of **T5** **conflicts** with the **read(B)** instruction of **T1**.
- Thus, we cannot move all the instructions of **T1** before those of **T5** by swapping consecutive non-conflicting instructions.
- However, the **final values** of accounts **A** and **B** after the execution of either **schedule 8** or the **serial schedule** are the **same** —\$960 and \$2040, respectively.

Testing for Conflict Serializability

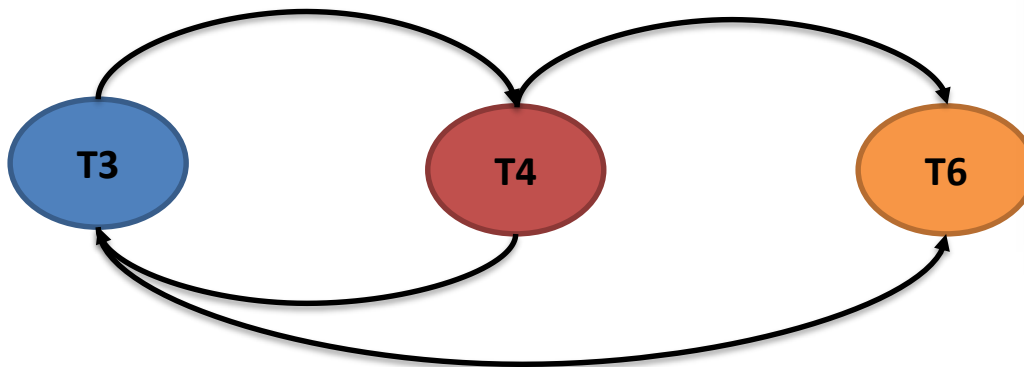
- Create a **precedence graph** $G(V,E)$ where V is the set of all the transactions.
- We will create an edge $T_i \rightarrow T_j$ if one of the **three** condition is **true**:

T_i	T_j
W(Q)	
	 R(Q)

T_i	T_j
R(Q)	
	 W(Q)

T_i	T_j
W(Q)	
	 W(Q)

- The **schedule S** is **serializable** if and only if the **precedence graph** $G(V,E)$ has no **cycles**.



T_3	T_4	T_6
read(Q)	write(Q)	
write(Q)		write(Q)

Schedule 9

Testing for Conflict Serializability

T_1	T_2
<code>read_item(X);</code> <code>X := X - N;</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>Y := Y + N;</code> <code>write_item(Y);</code>	 <code>read_item(X);</code> <code>X := X + M;</code> <code>write_item(X);</code>

Schedule A

T_1	T_2
<code>read_item(X);</code> <code>X := X - N;</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>Y := Y + N;</code> <code>write_item(Y);</code>	<code>read_item(X);</code> <code>X := X + M;</code> <code>write_item(X);</code>

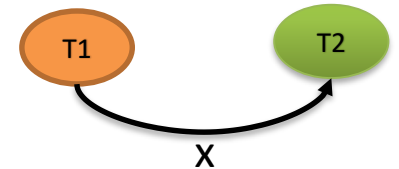
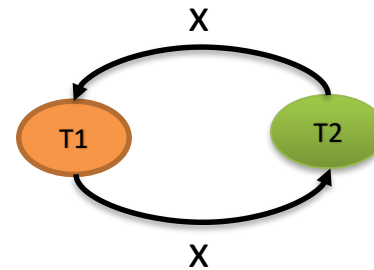
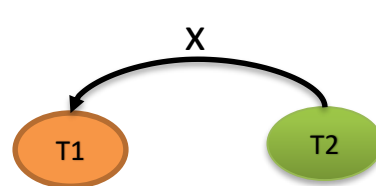
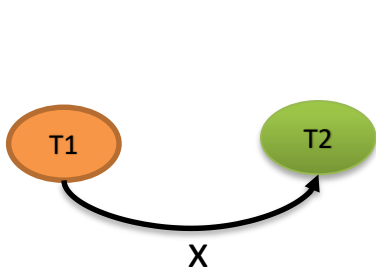
Schedule B

T_1	T_2
<code>read_item(X);</code> <code>X := X - N;</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>Y := Y + N;</code> <code>write_item(Y);</code>	 <code>read_item(X);</code> <code>X := X + M;</code> <code>write_item(X);</code>

Schedule C

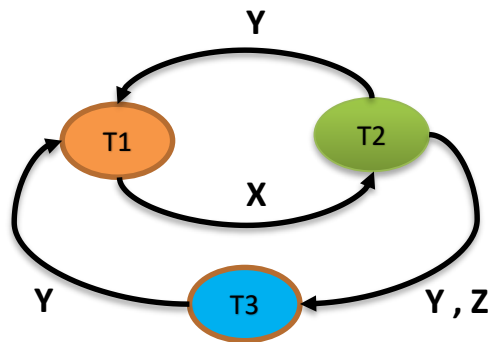
T_1	T_2
<code>read_item(X);</code> <code>X := X - N;</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>Y := Y + N;</code> <code>write_item(Y);</code>	 <code>read_item(X);</code> <code>X := X + M;</code> <code>write_item(X);</code>

Schedule D



Testing for Conflict Serializability

Transaction T_1	Transaction T_2	Transaction T_3
<code>read_item(X);</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>write_item(Y);</code>	<code>read_item(Z);</code> <code>read_item(Y);</code> <code>write_item(Y);</code> <code>read_item(X);</code> <code>write_item(X);</code>	<code>read_item(Y);</code> <code>read_item(Z);</code> <code>write_item(Y);</code> <code>write_item(Z);</code>
Schedule E		



Equivalent serial schedules

None

Reason

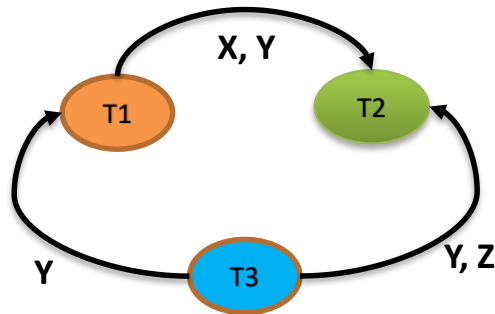
Cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$

Cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$

Testing for Conflict Serializability

Transaction T_1	Transaction T_2	Transaction T_3
<code>read_item(X);</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>write_item(Y);</code>	 <code>read_item(Z);</code> <code>read_item(Y);</code> <code>write_item(Y);</code> <code>read_item(X);</code> <code>write_item(X);</code>	<code>read_item(Y);</code> <code>read_item(Z);</code> <code>write_item(Y);</code> <code>write_item(Z);</code>

Schedule F

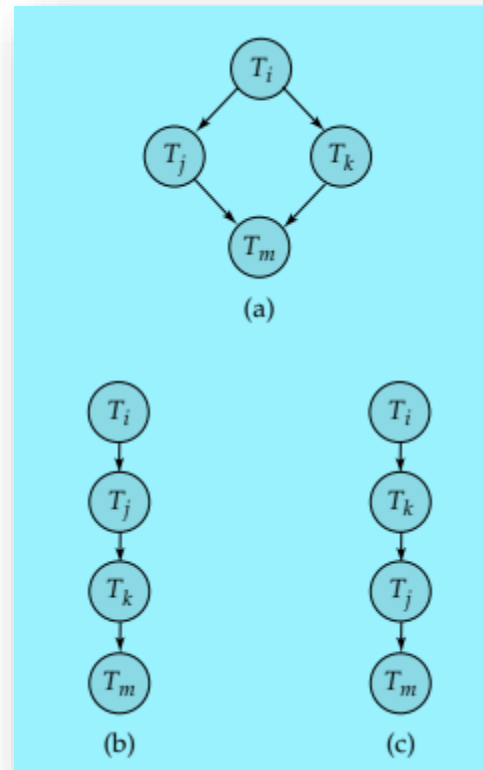


Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

Serializability Order

- A **Serializability order** of the **Transactions** can be obtained through **Topological Sorting**.



View Serializability

- **View equivalence:**
- Consider two **schedules S** and **S'** , where the same set of **transactions** participates in both schedules.
- The **schedules S** and **S'** are said to be **view equivalent** if **three conditions** are met:
- **Condition 1:**
- For each **data item Q**, if transaction **T_i** reads the **initial value of Q** in **schedule S**, then **transaction T_i** must, in **schedule S'** , also **read the initial value of Q**.

Schedule S	
T _i	T _j
Read(Q)	Read(Q)

Schedule S'	
T _i	T _j
Read(Q)	Read(Q)

View Serializability

- **Condition 2:**
- For each **data item Q**, if transaction **T_i** executes **read(Q)** in **schedule S**, and if that value was produced by a **write(Q) operation** executed by **transaction T_j**, then the **read(Q) operation of transaction T_i** must, in **schedule S'**, also read the value of **Q** that was produced by the same **write(Q) operation** of **transaction T_j**.

Schedule S	
T _i	T _j
Read(Q)	Write(Q)

Schedule S'	
T _i	T _j
Read(Q)	Write(Q)

View Serializability

- **Condition 3:**
- For each **data item Q**, the transaction (if any) that performs the **final write(Q)** operation in **schedule S** must perform the **final write(Q)** operation in **schedule S'**.

Schedule S	
Ti	Tj
Read(Q) Write(Q)	Write(Q)
Write(Q)	

Schedule S'	
Ti	Tj
Read(Q) Write(Q)	Read(Q) Write(Q)

View Serializability

- **Conditions 1** and **2** ensure that each **transaction reads** the same values in both schedules and, therefore, performs the same computation.
- **Condition 3**, coupled with **conditions 1** and **2**, ensures that **both schedules result** in the same **final system state**.

<i>T₁</i>	<i>T₂</i>
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>

<i>T₁</i>	<i>T₂</i>
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>

- **Schedule 1** is **not view equivalent** to **Schedule 2**, since, in **schedule 1**, the value of **account A** read by **transaction T2** was **produced by T1**, whereas this case does not hold in **Schedule 2**.

View Serializability

- However, **schedule 1** is **view equivalent** to **schedule 3**, because the values of account **A** and **B** read by transaction **T2** were produced by **T1** in both schedules.

T ₁	T ₂
read(A) $A := A - 50$ write(A) read(B) $B := B + 50$ write(B)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A) read(B) $B := B + temp$ write(B)

T ₁	T ₂
read(A) $A := A - 50$ write(A)	read(A) $temp := A * 0.1$ $A := A - temp$ write(A)
read(B) $B := B + 50$ write(B)	read(B) $B := B + temp$ write(B)

- The concept of **view equivalence** leads to the concept of **View Serializability**.
- We say that a **schedule S** is **view serializable** if it is **view equivalent** to a **serial schedule**.

View Serializability

- **Schedule 9** is **view serializable** as it is **view equivalent** to the **serial schedule** $\langle T_3, T_4, T_6 \rangle$, since the one **read(Q)** instruction reads the **initial value** of **Q** in **both schedules**, and **T6** performs the **final write** of **Q** in **both schedules**.

T_3	T_4	T_6
read(Q)		
write(Q)	write(Q)	
		write(Q)

Schedule 9

- Every **Conflict-serializable** schedule is **also View serializable**, but there are **View serializable schedules** that are **not Conflict serializable**.
- Indeed, **Schedule 9** is **not conflict serializable**, since every pair of consecutive instructions conflicts, and, thus, **no swapping of instructions is possible**.

View Serializability

T_3	T_4	T_6
read(Q)	write(Q)	
write(Q)		
		write(Q)

Schedule 9

- In **Schedule 9**, transactions **T4** and **T6** perform **write(Q)** operations without having performed a **read(Q)** operation.
- **Writes** of this sort are called **blind writes**.
- **Blind writes** appear in any **view-serializable schedule** that is **not conflict serializable**.

Testing for View Serializability

- **Testing** for **View Serializability** is rather **complicated**.
- In fact, it has been shown that the **problem of testing for view serializability** is itself **NP-complete**.
- Thus, **almost** certainly **there exists no efficient algorithm** to **test for View Serializability**.

Recoverable Schedules

- Consider **schedule 11** in Figure below, in which **T9** is a transaction that performs only one instruction: **read(A)**.

T ₈	T ₉
read(A)	
write(A)	
	read(A)
read(B)	

- Suppose that the system allows **T9 to commit** immediately after executing the **read(A)** instruction.
- Thus, **T9 commits** before **T8** does.
- Now suppose that **T8 fails** before it commits.
- Since **T9** has **read** the value of data item **A** written by **T8**, we must **abort T9** to ensure transaction **atomicity**.
- However, **T9** has **already committed** and **cannot be aborted**.

Recoverable Schedules

- Thus, we have a situation where it is **impossible to recover correctly from the failure of T8.**
- **Schedule 11**, with the **commit** happening **immediately after the read(A)** instruction, is an **example** of a **Non-recoverable schedule** , which **should not be allowed.**
- A **Recoverable schedule** is one where, for **each pair of transactions Ti and Tj** such that :
 - Tj reads a data item previously written by Ti.
 - The **Commit operation of Ti** appears **before the Commit operation of Tj** .

T ₈	T ₉
read(A)	
write(A)	
	read(A)
	write(A)
Commit	
	Commit

Cascadeless Schedules

- Sometimes to **recover** correctly from the failure of a transaction **T_i**, we may have to **roll back several transactions**.
- **Example:** Consider the partial schedule of Figure:
- **Transaction T₁₀** writes a value of **A** that is read by **transaction T₁₁**.
- **Transaction T₁₁** writes a value of **A** that is read by **transaction T₁₂**.
- Suppose that, at this point, **T₁₀ fails**.
- **T₁₀ must be rolled back**.
- Since **T₁₁ is dependent on T₁₀**, **T₁₁ must be rolled back**.
- Since **T₁₂ is dependent on T₁₁**, **T₁₂ must be rolled back**.
- This **phenomenon**, in which a **single transaction failure leads to a series of transaction rollbacks**, is called **cascading rollback**.

T ₁₀	T ₁₁	T ₁₂
read(A) read(B) write(A)	read(A) write(A)	read(A)

Cascadeless Schedules

- **Cascading rollback** is **undesirable**, since it leads to the **undoing** of a significant amount of work.
- It is **desirable to restrict** the schedules to those where **cascading rollbacks** cannot occur.
- Such schedules are called **Cascadeless Schedules**.
- Formally, a **Cascadeless Schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the **commit operation** of T_i appears **before** the **read operation** of T_j .
- **Every Cascadeless** schedule is also **Recoverable**.